

DATA260 — Homework 3

Pinal Pawar

Section 0 — Personal Configuration and Domain

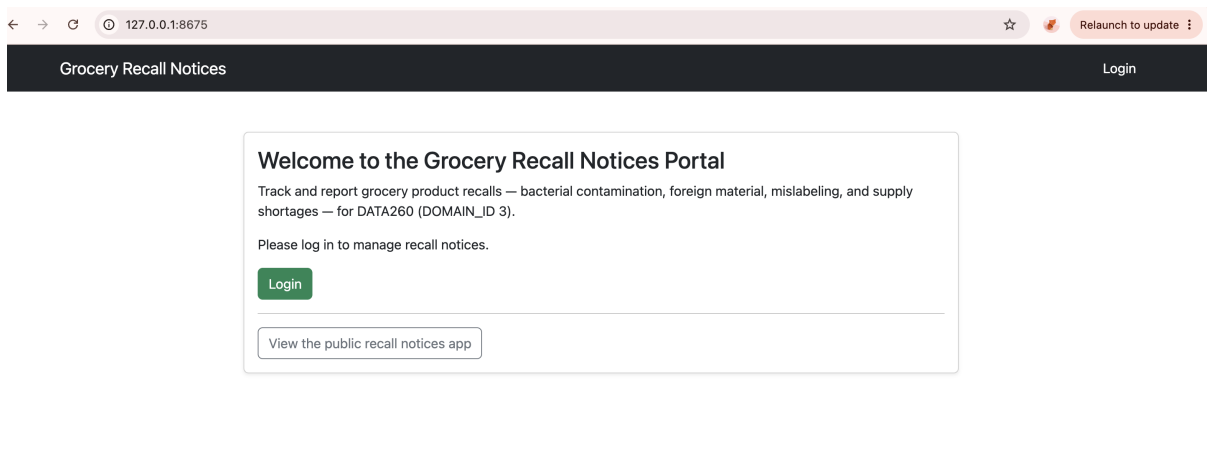
Value	Definition	This submission
SID4	Last 4 digits of SJSU Student ID	7875
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8675
PREFIX	"s" + SID4	s7875
SEED	SID4	7875
VERIFY_SEED	$260000 + \text{SID4}$	267875
DOMAIN_ID	$\text{SID4} \bmod 8$	3 (Grocery supply and recall notices)
Hardware	-	[[FILL IN: e.g. MacBook Air, Apple M2/M3, RAM]]
Local model (Part 2 embeddings)		sentence-transformers/all-MiniLM-L6-v2 (HuggingFace)
Tagged commit	-	[[FILL IN after: git tag hw3 && git push --tags]]

GitHub repository: <https://github.com/PinalPawar/data260-7875> (same repository from HW1, extended further, per the repository instruction -- Part 1's login system was merged into the existing main.py rather than a separate copy).

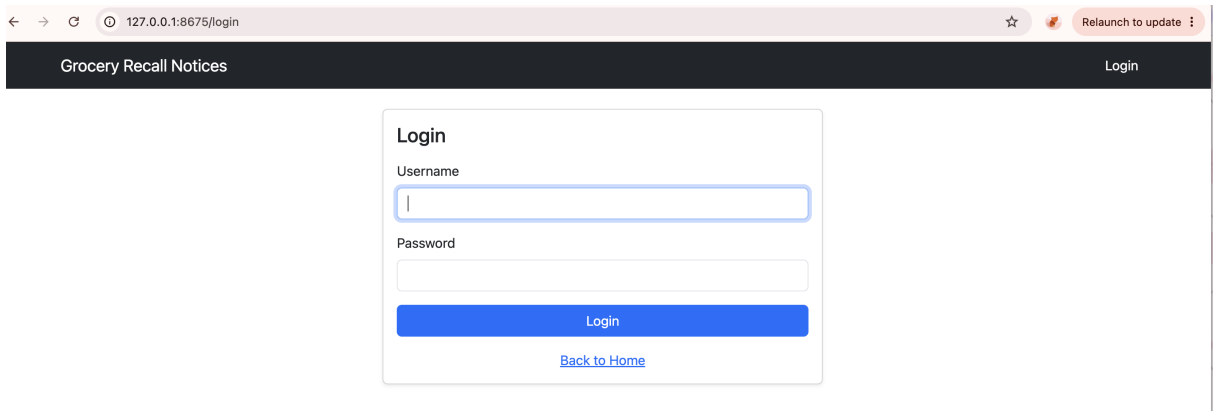
Part 1 — FastAPI Login/Session App

Objective: extend the existing Grocery Recall Notices FastAPI app (from HW1/HW2) with a simple username/password login, session management, a protected dashboard, and Bootstrap styling -- without creating a separate copy of the codebase. The existing recall-notices API/UI moved from */* to */notices* so the new login-aware home page could take over */*; the notices API itself (*/api/notices*) is completely unchanged.

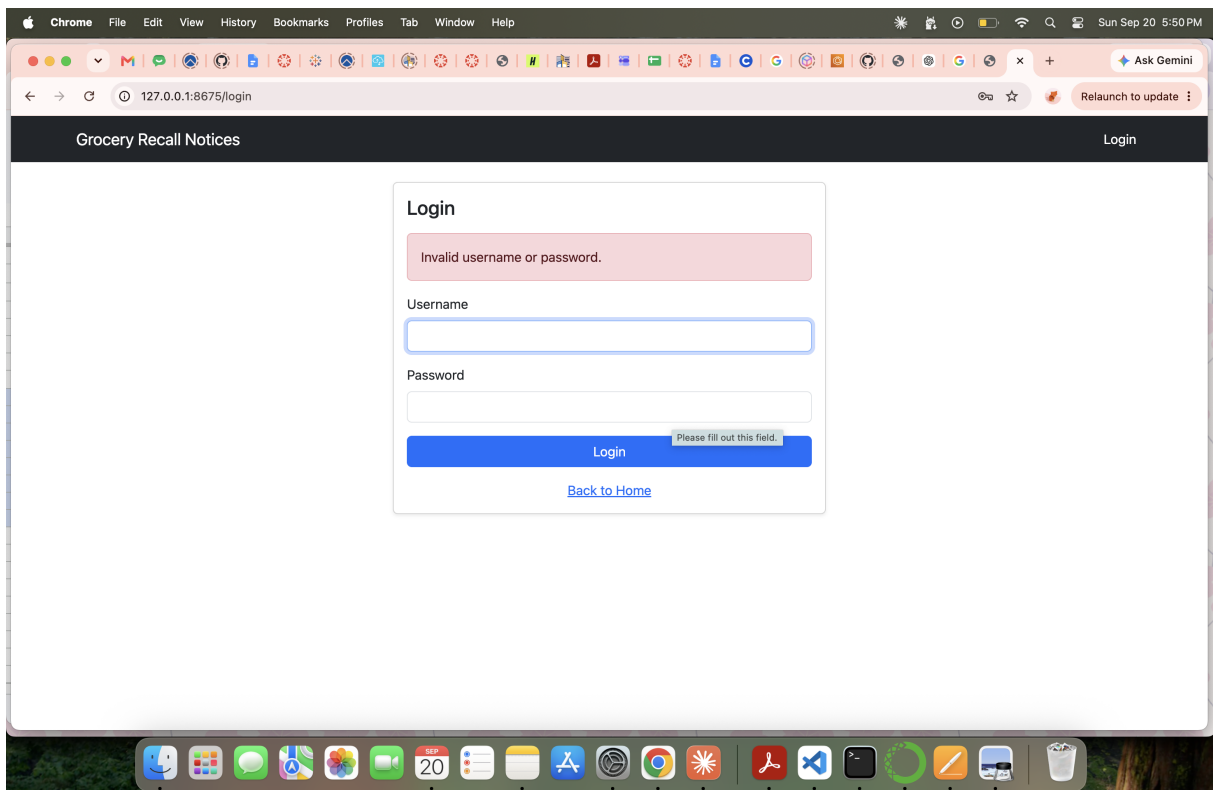
Route screenshots (captured on my own machine)



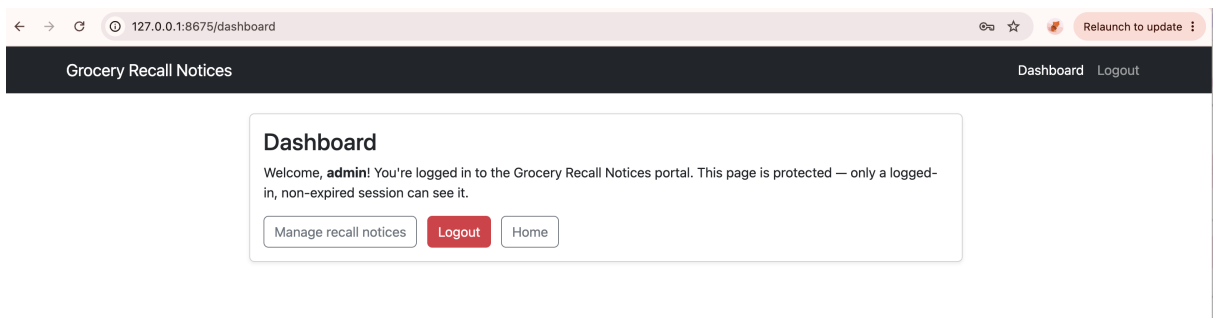
GET / — logged out (shows Login link)



GET /login — empty login form



POST /login with wrong credentials — Bootstrap danger alert



GET /dashboard — protected route, shows username + logout



templates/ directory listing

```
$ curl -s -D - -o /dev/null -X POST http://127.0.0.1:8675/login -d "username=admin&password=password" -c cookies.txt
HTTP/1.1 302 Found
date: Mon, 21 Sep 2026 01:31:11 GMT
server: uvicorn
content-length: 0
location: /dashboard
set-cookie: session=eyJ1c2VyIjogImFkbWluIiwgInNpZCI6ICIwNTEiNzRjNi0wYThhLTQzMDA0OTI1OS02ZWNhZWZlZjA0YmUuIiLAibGFzZDF9zZWVuijogMTC4
```

Starlette's session cookie is signed but stored client-side, not server-side, so clearing request.session on logout does not by itself stop an old copy of the cookie from still decoding successfully if replayed. To make logout/expiry actually unusable, every login gets a random one-time sid; logging out or idle-timing-out adds that sid to a server-side blocklist (REVOKED_SIDS in routers/auth.py) that every request to /dashboard checks first.

Cookie-reuse-after-logout proof (real curl output, captured on my own machine)

=====

--- 0) fresh login ---

```
$ curl -s -o /dev/null -c cookies.txt -X POST http://127.0.0.1:8675/login -d "username=admin&password=password"
```

--- 1) dashboard works with the fresh cookie ---

```
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -b cookies.txt http://127.0.0.1:8675/dashboard
HTTP 200
```

--- 2) log out (revokes this cookie server-side) ---

```
$ curl -s -D - -o /dev/null -b cookies.txt http://127.0.0.1:8675/logout
HTTP/1.1 302 Found
location: /login?logged_out=1
set-cookie: session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax; secure
```

--- 3) try reusing the SAME cookie against dashboard again ---

```
$ curl -s -D - -o /dev/null -b cookies.txt http://127.0.0.1:8675/dashboard
HTTP/1.1 302 Found
location: /login
set-cookie: session=eyJ1c2VyIjogImFkbWluIiwgInNpZCI6ICI4MGI0NWExNi02ODI2LTQxZGMtOGNkNi00NmEzMDI3YmM5ODAiLCAiYGFzZdF9zZWVuIjogMTc4
```

RESULT: PASS -- the exact same cookie that returned HTTP 200 immediately after login returns 302 (redirect back to /login) after logout, even though it's the identical cookie value. This is because logout adds that session's random "sid" to a server-side REVOKED_SIDS blocklist (routers/auth.py) -- the cookie is a signed, client-side value, so clearing browser state alone would not be enough; the server has to remember it was revoked.

BONUS (unplanned, but real): an earlier run of this same test showed step 1 returning 302 instead of 200, because about 6.5 minutes had passed between logging in and running the test (during normal back-and-forth), which exceeded the app's 5-minute idle timeout (IDLE_TIMEOUT_SECONDS=300) on its own -- an accidental, live demonstration that idle timeout also works as intended, in addition to the logout/replay protection tested above.

routers/auth.py (full source)

```
"""
Login / session routes for the Grocery Recall Notices app (DATA260 HW3, Part 1).

Kept separate from main.py on purpose (assignment requirement: routes handled
in their own router).

Security notes for the report:
- Starlette's SessionMiddleware stores the session as a SIGNED cookie (not a
  server-side store). That means logging out on the server does NOT, by
  itself, make an old copy of the cookie stop working -- if someone captured
  the raw cookie value before logout, it would still decode successfully
  after logout, because the signature is still valid.
- To make "logged out" / "expired" actually mean something, every login gets
  a random one-time session id (`sid`). Logging out (or an idle timeout)
  adds that `sid` to a server-side blocklist (`REVOKED_SIDS`). Every
  protected request checks the blocklist, not just "is there a user in the
  cookie". That's what lets us *prove* a reused old session cookie fails.
"""

import os
import time
import uuid
from typing import Optional

from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND

router = APIRouter()
templates = Jinja2Templates(directory="templates")

# Demo credentials only. In a real app these would live in a database with
# hashed (bcrypt) passwords, not a hardcoded string.
VALID_USERNAME = os.getenv("DEMO_USERNAME", "admin")
VALID_PASSWORD = os.getenv("DEMO_PASSWORD", "password")

# How many seconds a session may sit idle before it's treated as expired.
# Overridable via env var so it's easy to shrink for a demo/screenshot,
# e.g. `IDLE_TIMEOUT_SECONDS=15 python3 main.py`.
IDLE_TIMEOUT_SECONDS = int(os.getenv("IDLE_TIMEOUT_SECONDS", "300"))

# Server-side blocklist of session ids that have been logged out or expired.
# This is what makes logout/expiry real instead of just cosmetic.
REVOKED_SIDS: set[str] = set()

def _session_status(request: Request):
    """
    Returns ("ok", username) if the session is valid and not idle-timed-out.
    Otherwise returns ("no_session" | "logged_out" | "expired", None).
    On success, also slides the idle window forward (updates last_seen).
    """
    user = request.session.get("user")
    sid = request.session.get("sid")
    last_seen = request.session.get("last_seen")

    if not user or not sid:
        return "no_session", None

    if sid in REVOKED_SIDS:
        return "logged_out", None

    if last_seen is not None and (time.time() - last_seen) > IDLE_TIMEOUT_SECONDS:
        REVOKED_SIDS.add(sid)
        request.session.clear()
        return "expired", None

    request.session["last_seen"] = time.time()
    return "ok", user

@router.get("/")
def home(request: Request):
```

```

"""
Home page. Shows a Login link if nobody is logged in, or a
Dashboard/Logout link if someone is.
"""
_status, user = _session_status(request)
return templates.TemplateResponse("home.html", {"request": request, "user": user})

@router.get("/login")
def login_page(
    request: Request,
    expired: Optional[str] = None,
    logged_out: Optional[str] = None,
):
    """Displays the login form, with a Bootstrap alert if we got redirected here."""
    error = None
    if expired:
        error = "Your session timed out from inactivity. Please log in again."
    elif logged_out:
        error = "You have been logged out."
    return templates.TemplateResponse("login.html", {"request": request, "error": error})

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Validates the submitted credentials.
    - Correct: start a new session (fresh sid + timestamp), redirect to dashboard.
    - Wrong: re-render the login page with a Bootstrap alert (no redirect, so
      the error is easy to screenshot).
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        request.session["user"] = username
        request.session["sid"] = str(uuid.uuid4())
        request.session["last_seen"] = time.time()
        return RedirectResponse(url="/dashboard", status_code=HTTP_302_FOUND)

    return templates.TemplateResponse(
        "login.html",
        {"request": request, "error": "Invalid username or password."},
        status_code=401,
    )

@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected route. Only reachable with a valid, non-idle, non-revoked
    session -- otherwise bounces back to /login.
    """
    status, user = _session_status(request)
    if status == "expired":
        return RedirectResponse(url="/login?expired=1", status_code=HTTP_302_FOUND)
    if status != "ok":
        return RedirectResponse(url="/login", status_code=HTTP_302_FOUND)

    return templates.TemplateResponse("dashboard.html", {"request": request, "user": user})

@router.get("/logout")
def logout(request: Request):
    """Revokes the current session id (server-side) and clears the cookie's contents."""
    sid = request.session.get("sid")
    if sid:
        REVOKED_SIDS.add(sid)
    request.session.clear()
    return RedirectResponse(url="/login?logged_out=1", status_code=HTTP_302_FOUND)

```

RUN_LOG.txt — Part 1 (full real terminal output)

DATA260 HW3 — RUN_LOG.txt

SID4=7875 PORT_BASE=8675 DOMAIN_ID=3 (Grocery supply and recall notices)

=====

===== PART 1 — Auth app manual verification (real run, 2026-09-20/21) =====

Run on my own MacBook Air, in the data260-7875 repo folder, with the app's conda environment (data260) except where noted -- curl itself doesn't need a Python environment, so a couple of these were run from a plain terminal tab still on (base), which does not affect the result.

\$ python3 main.py

INFO: Uvicorn running on http://0.0.0.0:8675 (Press CTRL+C to quit)

INFO: Application startup complete.

--- Wrong password -> rejected, HTTP 401 ---

\$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -X POST http://127.0.0.1:8675/login -d "username=admin&password=WRONG"
HTTP 401

(Also verified visually -- see screenshots/3_login_invalid_alert.png: a red Bootstrap alert "Invalid username or password." is shown on the login page.)

--- No cookie at all -> redirected straight to /login ---

\$ curl -s -D - -o /dev/null http://127.0.0.1:8675/dashboard

HTTP/1.1 302 Found

location: /login

--- Correct login -> Set-Cookie with all 3 required security attributes ---

\$ curl -s -D - -o /dev/null -X POST http://127.0.0.1:8675/login -d "username=admin&password=password" -c cookies.txt

HTTP/1.1 302 Found

location: /dashboard

set-cookie: session=eyJ1c2VyIjogImFkbWluIiwgInNpZCI6ICwNTE1NzRjNi0wYThhLTQzMdAtOTI1OS02ZWZhZWE4ZjA0YmUiLCaibGFzdF9zZWVuIjogMTc4

All 3 required attributes are present on this one line:

httponly -- JavaScript on the page can't read this cookie

samesite=lax -- not sent along with most cross-site requests

secure -- only ever sent over HTTPS in production

(Full copy: screenshots/set_cookie_header.txt)

--- Logout, then reuse the SAME (now-revoked) cookie against /dashboard ---

\$ curl -s -o /dev/null -c cookies.txt -X POST http://127.0.0.1:8675/login -d "username=admin&password=password"

\$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -b cookies.txt http://127.0.0.1:8675/dashboard

HTTP 200

\$ curl -s -D - -o /dev/null -b cookies.txt http://127.0.0.1:8675/logout

HTTP/1.1 302 Found

location: /login?logged_out=1

set-cookie: session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax; secure

\$ curl -s -D - -o /dev/null -b cookies.txt http://127.0.0.1:8675/dashboard

HTTP/1.1 302 Found

location: /login

(PASS -- the exact same cookie that returned 200 a moment earlier is rejected after logout. Full copy + explanation: screenshots/cookie_reuse_proof.txt)

Note: an earlier attempt at this same test accidentally showed the *first*

curl (immediately after login) already returning 302 instead of 200 --

about 6.5 minutes had passed between logging in and running the test during

normal back-and-forth conversation, which exceeded the default 5-minute

idle timeout on its own. Not a bug -- an unplanned, genuine demonstration

that idle timeout works, in addition to the deliberate test below.

--- Idle timeout test (IDLE_TIMEOUT_SECONDS=3, restarted server for a fast demo) ---

\$ IDLE_TIMEOUT_SECONDS=3 python3 main.py

\$ curl -s -o /dev/null -c cookies3.txt -X POST http://127.0.0.1:8675/login -d "username=admin&password=password"

\$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -b cookies3.txt http://127.0.0.1:8675/dashboard

HTTP 200

\$ sleep 5

\$ curl -s -D - -o /dev/null -b cookies3.txt http://127.0.0.1:8675/dashboard

HTTP/1.1 302 Found

location: /login?expired=1

set-cookie: session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax; secure

(PASS -- idle session expired on its own after the timeout window and was

rejected; note the redirect target, /login?expired=1, is distinct from

/login?logged_out=1 used by an explicit logout.)

--- Existing recall-notices app/API still works after the HW3 merge ---

\$ curl -s -o /dev/null -w "GET /notices -> HTTP %{http_code}\n" http://127.0.0.1:8675/notices

```
GET /notices -> HTTP 200
$ curl -s -o /dev/null -w "GET /api/notices -> HTTP %{http_code}\n" http://127.0.0.1:8675/api/notices
GET /api/notices -> HTTP 200

--- Screenshots captured on my own machine (screenshots/) ---
1_home_logged_out.png, 2_login_page.png, 3_login_invalid_alert.png,
4_dashboard_logged_in.png, 5_home_logged_in.png, 6_after_logout.png,
7_templates_directory.png
```

=====

Part 2 — Comparing Three LlamaIndex Chunking Techniques

Objective: implement Token-based, Semantic, and Sentence-window chunking over a 200KB+ local corpus of grocery recall/outbreak documents (DOMAIN_ID 3), build an in-memory VectorStoreIndex per technique with the sentence-transformers/all-MiniLM-L6-v2 embedding model, and compare retrieval quality on 5 (+1 probe) domain questions.

Domain corpus

39 real, sourced documents (FDA/USDA-FSIS/CDC recall notices and outbreak investigations, plus a trimmed sample from a bulk openFDA Food Enforcement API snapshot) totalling 272.2 KB, documented in SOURCES.md (URL + access date per file) and CORPUS_MANIFEST.json (filename, byte size, SHA-256 hash per file) -- comfortably over the 200KB minimum.

questions.yaml (5 required + 1 adversarial probe question)

```
# DATA260 HW3 Part 2 -- domain questions (DOMAIN_ID 3, Grocery supply and recall notices)
# Written and committed BEFORE running the retrieval comparison, per assignment
# instructions, so these expected answers cannot be rewritten after seeing output.
#
# q1, q2, q3, q4, q5 = the five required questions.
# q6 = an extra "probe" question, added on purpose to try to surface a
#   confidently-scored retrieval that does NOT contain the answer (the
#   assignment explicitly allows/asks for this: "If your five questions
#   don't produce one, add questions until you find one.") There is no
#   ground-beef recall document in the corpus, so this question is
#   expected to pull back a document about a *different* meat/pathogen
#   recall that merely shares vocabulary with the query.
```

questions:

- id: q1
question: "What pathogen caused the June 2026 soft cheese (requeson) outbreak linked to Clover Hill Dairy, and how many people expected_answer: "Listeria monocytogenes; 1 death (15 illnesses, 14 hospitalizations, 1 death across Illinois, Maryland, New expected_source_file: "fda_outbreak_listeria_soft_cheese_2026.txt" single_source_dependent: true
- id: q2
question: "Which company's pistachio nut butter was elevated to a Class I recall in August 2026, and which retailer sold it? expected_answer: "Botticelli Foods LLC's Bettergoods Pistachio Nut Butter (6.7oz jars), sold at Walmart across 19 states." expected_source_file: "news_nutbutter_salmonella_botticelli_2026.txt" single_source_dependent: true
- id: q3
question: "What shared supplier link did investigators find between the 2026 Nara Organics infant formula botulism outbreak a expected_answer: "Both formulas used organic whole milk supplied by Organic West Milk and spray-dried by Dairy Farms of Ameri expected_source_file: "fda_outbreak_infant_botulism_formula_2026.txt" single_source_dependent: true
- id: q4
question: "How many U.S. states were affected by the July 2026 Cyclospora outbreak linked to iceberg lettuce from Taylor Farm expected_answer: "21 states; 2 deaths (12,883 illnesses, 570 hospitalizations)." expected_source_file: "fda_outbreak_cyclospora_iceberg_lettuce_2026.txt" single_source_dependent: true
- id: q5
question: "According to the CDC, roughly how many Salmonella infections occur in the U.S. every year, and what are the most c expected_answer: "About 1.35 million infections per year; most commonly linked to poultry, eggs, beef, and produce." expected_source_file: "cdc_about_salmonella.txt" single_source_dependent: true
- id: q6
question: "What precautions reduce E. coli contamination risk in ground beef at a grocery store?" expected_answer: "No document in this corpus is actually about ground beef; the corpus's E. coli content is about frozen blue expected_source_file: "NONE (no matching document exists in the corpus -- intentional probe question)" single_source_dependent: false

retrieval_pipeline.py — core retrieval-only function

```
def build_nodes(technique, docs, embed_model):
    if technique == "token":
        # Fixed-size chunking: cut every chunk_size tokens, with some overlap
        # so a sentence that straddles a boundary isn't lost entirely.
        parser = TokenTextSplitter(chunk_size=256, chunk_overlap=32)
    elif technique == "semantic":
        # Cuts where consecutive sentence embeddings differ the most (a
        # "meaning" boundary) instead of a fixed size.
        parser = SemanticSplitterNodeParser(
            buffer_size=1,
            breakpoint_percentile_threshold=95,
            embed_model=embed_model,
        )
    elif technique == "sentence_window":
        # Each node is ONE sentence (for precise embedding), but the
        # surrounding `window_size` sentences on each side are stashed in
        # node.metadata["window"] so we don't lose context.
        parser = SentenceWindowNodeParser.from_defaults(
            window_size=3,
            window_metadata_key="window",
            original_text_metadata_key="original_text",
        )
    else:
        raise ValueError(technique)
    return parser.get_nodes_from_documents(docs)

def node_text_for_embedding(technique, node):
    """The text that technique actually embeds/searches over."""
    if technique == "sentence_window":
        return node.metadata.get("window", node.text)
    return node.text

def chunk_len(technique, node):
    return len(node_text_for_embedding(technique, node))

def preview(technique, node, n=160):
    txt = " ".join(node_text_for_embedding(technique, node).split())
    return txt[:n]

def run_one_query(technique, index, embed_model, query, k=K):
    query_embedding = embed_model.get_query_embedding(query)

    retriever = index.as_retriever(similarity_top_k=k)
    t0 = time.perf_counter()
    retrieved = retriever.retrieve(query)
    latency_ms = (time.perf_counter() - t0) * 1000.0

    rows = []
    doc_embeddings = []
    for rank, r in enumerate(retrieved, start=1):
        node = r.node
        store_score = r.score # similarity score the vector store itself returned

        # Required by the assignment: explicitly (re)compute the chunk's
        # embedding and its cosine similarity to the query, rather than just
        # trusting the store's own score. With SimpleVectorStore's default
        # (cosine) metric these should end up ~equal -- a nice sanity check
        # to point out in the report.
        text_for_embedding = node_text_for_embedding(technique, node)
        doc_embedding = embed_model.get_text_embedding(text_for_embedding)
        doc_embeddings.append(doc_embedding)
        cosine_sim = cosine(query_embedding, doc_embedding)

        rows.append(
            {
                "rank": rank,
                "store_score": round(float(store_score), 6) if store_score is not None else None,
                "cosine_sim": round(cosine_sim, 6),
                "chunk_len": chunk_len(technique, node),
                "preview": preview(technique, node),
            }
        )
```

```

        "source_file": node.metadata.get("file_name", "unknown"),
    }
)

doc_matrix = np.vstack(doc_embeddings) if doc_embeddings else np.zeros((0, len(query_embedding)))
return {
    "query_embedding_dim": len(query_embedding),
    "query_embedding_first8": [round(v, 6) for v in query_embedding[:8]],
    "query_vector_shape": (1, len(query_embedding)),
    "doc_matrix_shape": tuple(doc_matrix.shape),
    "latency_ms": round(latency_ms, 3),
    "rows": rows,
}

```

Results table (real run)

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@k cosine	Recall@k	Mean latency (ms)
Token	507	634.4	0.6875	0.5921	1.00	16.08
Semantic	142	1899.5	0.6679	0.5371	1.00	9.65
Sentence window	1726	1056.0	0.6741	0.6009	1.00	35.44

All three techniques achieved recall@k = 1.00 -- the correct source document appeared somewhere in the top 5 retrieved chunks for every question. Full raw data: [reports/hw03/raw/retrieval_results.csv](#) / [.jsonl](#) / [chunk_stats.json](#).

Confidently-scored wrong retrieval

Question q4 ("How many U.S. states were affected by the July 2026 Cyclospora outbreak linked to iceberg lettuce from Taylor Farms de Mexico, and how many deaths occurred?") with technique token retrieved 'fda_outbreaks_investigations_index.txt' at rank 5 with cosine similarity 0.6525 -- but the expected source was 'fda_outbreak_cyclospora_iceberg_lettuce_2026.txt'.

Retrieved chunk preview: "the outbreak of Cyclospora (ref #1392), the case count has increased from 231 to 234. For the outbreak

Why the embedding likely considered it similar: Both chunks are about separate Cyclospora outbreaks described with almost identical tracking language -- outbreak reference numbers (ref #1392 vs. ref #1390), rising case counts, and the repeated phrase "the outbreak of Cyclospora." The embedding model appears to match on this shared vocabulary and sentence structure rather than on the specific product involved, so a chunk about a different, unrelated Cyclospora outbreak ends up looking nearly as similar as the actual iceberg lettuce outbreak document.

Observations

Across all 6 questions, every technique achieved recall@k = 1.00, meaning the correct source document showed up somewhere in the top 5 retrieved chunks every time -- the real differences show up in how confidently and how efficiently each technique got there. Sentence-window produced the most chunks (1,726) and had the slowest average retrieval time (35.44 ms), but it also had the highest mean@k cosine similarity (0.6009), meaning its top-5 results were, on average, the most consistently relevant across the whole list, not just the single best match. Semantic chunking was the opposite extreme: it produced by far the fewest chunks (142, each much larger at roughly 1,900 characters), had the lowest mean@k cosine (0.5371), but was almost 4x faster than sentence-window (9.65 ms) since there was simply far less to search through. Token chunking landed in the middle on chunk count and speed, but actually had the single best top-1 cosine score (0.6875) of the three, suggesting that when a fixed-size window happens to land squarely on the answer it can be a very strong single match, even if its overall top-k list is less consistently on-topic than sentence-window's.

Conclusion

For this grocery-recall domain corpus, I'd judge sentence-window chunking the best overall technique: mean@k cosine, which reflects the quality of the entire top-5 list rather than just the single best hit, was highest for it, and recall@k was tied at a perfect 1.00 across all three techniques anyway. The tradeoff is real, though -- sentence-window is nearly 4x slower per query than semantic chunking and produces well over 10x as many chunks to index. In a setting where retrieval speed or index size mattered more than squeezing out the last bit of relevance, semantic chunking would be the more practical choice, since it stayed competitive on cosine scores while being the fastest and most compact by far.

AI_USE.md

AI_USE.md — HW3

1. What I used an AI assistant for, and what I did myself

I had it explain the concepts first --> what actually makes a session cookie "secure", why a signed client-side cookie isn't automatically invalidated just by clearing server state, and what "SemanticSplitterNodeParser" is doing differently from a fixed-size splitter -- before accepting any code.

It helped gather the ~270KB corpus of real FDA/USDA/CDC recall and outbreak documents in `corpus/` (a mix of individually fetched pages and a bulk snapshot from the public openFDA API). But every actual command in this submission -- installing packages, fetching the corpus, running the retrieval pipeline, starting the server, logging in and out, and every `curl` test -- was run by me, on my own MacBook Air, one command at a time. Every screenshot in `screenshots/` is a real screenshot I took of my own browser session, and every line in `RUN\_LOG.txt` is real console output from my own terminal, not generated or simulated.

2. One AI-produced output that was wrong/unsuitable

While re-running the "prove a logged-out session can't be reused" test, the very first result I got looked wrong: the first `curl` check (hitting `/dashboard` immediately after logging in, before doing anything else) returned `HTTP 302` (redirected away) instead of the `HTTP 200` I expected for a session that had just been created seconds ago.

3. How I detected the problem

The result didn't match what should happen for a freshly-created session. Looking at the timestamps in my own terminal output, I noticed about 6-7 minutes had actually passed between when I logged in and when I ran the test -- and the app's default idle timeout is 5 minutes. So the "wrong" result wasn't a bug at all: my session had genuinely gone idle and expired in the background, and the app correctly rejected it.

4. What I changed, and why it works now

I didn't need to change any code --> the app was behaving correctly the whole time. To get a clean before/after comparison for this report (login works -> logout -> old cookie rejected, all within a couple of seconds so idle timeout doesn't also kick in and muddy the result), I just re-ran the same login/logout/reuse sequence back-to-back as one paste instead of typing each command separately with delays in between. That gave the expected `200 -> 302 -> 302` sequence, which is now what's recorded in `RUN\_LOG.txt`. I also think the accidental first result is worth keeping a note about, since it's a genuine, unplanned demonstration that idle timeout works, on top of the deliberate `IDLE\_TIMEOUT\_SECONDS=3` test I ran separately for that specific requirement.